

PROGRAMOWANIE NISKOPOZIOMOWE

NOTATKI

„Mam nadzieję, że nie będzie na niskim poziomie.”
„To było na systemach operacyjnych.”

MACIEJ MIKOŁAJCZAK
Kraków, 2025

Spis treści

1. Architektura x86	2
2. ABI Linuxa	3
3. ABI C	5
4. ABI C++	7

1. Architektura x86

2025-02-26

Procesor posiada pamięć wewnętrzną – rejestry. Są to pojedyncze komórki pamięci, są potrzebne, bo trzeba mieć jakąś minimalną pamięć potrzebną do komunikacji z RAM'em.

Pierwotnie były rejestry **ax**, **bx**, **cx**, **dx** i rejestry specjalne **si**, **di**, **sp**, **bp** – tak było w architekturze 16-bitowej.

Potem zmieniono na 32-bitowe – dodajemy do nazwy na początek **e** jak extended.

Na 64-bitach zamiast **e** jest **r** i dodano 8 nowych rejestrów – **r8**, **r9**, ..., **r15**.

Ostatecznie jest **rax**, **rbx**, **rcx**, **rdx**, **rsi**, **rdi**, **rsp**, **rbp**, **r8**, ..., **r15**.

Stare nazwy dalej obowiązują – 32 dolne bity **rax** funkcjonuje pod nazwą **eax**, a dolne 16 – **ax**, do tego jest jeszcze **ah**, **al** – górna i dolna połowa **ax**, tak było od początku.

W rejestrach specjalnych wygląda to tak samo, nie ma podziału na części 8-bitowe, jest tylko dolna – **si** -> **sil**, nawet ona nie jest dostępna zawsze, tylko w niektórych instrukcjach.

W najnowszych rejestrach mamy nową nomenklaturę:

- **b** – jeden bajt
- **w** – słowo, dwa bajty (bo kiedyś była architektura 16-bitowa)
- **d** – double word, cztery bajty, bo to podwójne słowo gdy rozumiemy słowo jako 16 bajtów
- **q** – quad word, osiem bajtów

r9 -> **r9d**, **r9w**, **r9b** dają nam dostęp do dolnych części rejestru o danej długości.

Tryb pracy procesora może oznaczać ilość bitów używanych do adresowania pamięci lub ilość bitów rejestru. Standardowo myślimy o tym pierwszym. W procesorze 16-bitowym mamy maksymalnie 2^{16} adresów, to jest dość mało, dlatego wprowadzono mechanizm segmentacji – mamy 16 bitów segmentu i 16 offsetu, to znacząco zwiększa ilość dostępnej pamięci. Jako, że segment rzadko się zmienia, to można ustalić segment i dalej pracować na wskaźniku 16-bitowym, segment zmienić jak będzie to potrzebne. Dlatego będą do tego osobne instrukcje.

Początkowo pamięć nie była chroniona – operowano na adresach fizycznych, więc każdy mógł zmienić pamięć. Wzmocniono segmentację – system operacyjny ustala, gdzie programy widzą swoje segmenty i upewnia się, że nikt nie odwołuje się poza swoją pamięć. To się stało już, gdy były procesory 32-bitowe.

Drugi mechanizm – adresy wirtualne, adres ma normalnie 32 bity, dzielimy je na 12 dolnych i resztę – ta reszta oznacza stronę pamięci, która ma 2^{12} bitów. System operacyjny ustala to, do czego w pamięci fizycznej odnosi się strona. Dzięki temu mamy możliwość chronienia pamięci, jak i jej współdzielenia.

W ten sposób otrzymujemy następujące tryby pracy procesora:

- Legacy Mode:
 - Real Mode – 16 bitów, prosta segmentacja, nie chroniona pamięć, 1MB dostępnej pamięci.
 - Protected Mode – 32 bity, segmentacja, stronicowanie, chroniona pamięć, 4GB pamięci.
 - Virtual-8086 – uruchamianie 16-bitowych programów w trybie Protected.
- Long Mode:
 - 64-Bit Mode – brak segmentacji (bo już możemy adresować dużo pamięci), za to stronicowanie, chroniona pamięć, rejestry i adresowanie 64-bitowe.
 - Compatibility Mode – dla użytkownika działa jak Protected Mode, dla systemu jak 64-Bit.

W Legacy i Compatibility Mode mamy do dyspozycji kilka różnych segmentów, w 64-Bit Mode zostają nam tylko trzy – CS odpowiedzialny za przechowywanie atrybutów (np. uprawnień), FS i GS używane do robienia rzeczy thread-local.

W instrukcjach będziemy używać różnych sposobów adresowania:

- Absolute – pełny adres podawany w instrukcji.
- ModR/M:

$$\text{address} = \text{base} + \text{index} \cdot \text{scale} + \text{displacement},$$

gdzie base i index są w rejestrach, $\text{scale} \in \{0, 1, 2, 4, 8\}$ i displacement są podane w instrukcji. To pozwala nam na ładne przechodzenie po pamięci.

- Instruction-relative:

$$\text{address} = \text{RIP} + \text{displacement},$$

czyli przesuwamy się od obecnej instrukcji, to pozwala umieszczać dane wspólnie z kodem.

- Implicit – adres jest wyznaczony przez typ instrukcji.

W Assemblerze nie mamy typów zmiennych, tylko ciągi bitów. Ich interpretacja jest zawarta w instrukcji. Instrukcja identyfikuje rozmiar (byte, word, doubleword, quadword, double quadword), położenie (stała podana w instrukcji, rejestr, adres w pamięci, port I/O) i interpretację (signed int, unsigned int, BCD – Binary Coded Decimal, Packed BCD digits, strings, floats).

BCD trzyma każdą cyfrę w kolejnym bajcie, ale właściwie są do tego potrzebne 4 bity, dlatego Packed BCD trzyma po dwie cyfry w bajcie.

Pracujemy na wartościach wielobajtowych, można te bajty pisać od najmniej lub najbardziej znaczących. Jeśli wpisujemy „po ludzku” – od lewej do prawej, to najbardziej znaczący bajt ma najmniejszy adres. Dlatego drugi tryb też ma sens.

Little endian – pod najniższym adresem jest najniższy bajt, tak działa x86 i amd64.

Big endian – pod najniższym adresem jest najwyższy bajt, np. PlayStation i Xbox, protokoły sieciowe.

Wielka lista instrukcji:

- Transfer danych – `mov`, `cmovXX`, `push`, `pop`.
- Arytmetyczne – `add`, `sub`, `neg`, `mul`, `imul` (mnożenie naturalne i całkowite), `div`, `idiv`, `dec`, `inc`, `lea`.
- Logiczne i bitowe – `and`, `or`, `xor`, `not`, `shl`, `shr`.
- Testowanie i porównania – `cmp`, `test`, `bt`, `setXX`.
- Sterowanie przepływem – `jmp`, `jXX`, `loop`, `call`, `ret`.
- Wywołania systemowe – `syscall`, `sysret`, `sysenter`, `sysexit`.

Rejestr znaczników to rejestr, którego pojedyncze bity mają pewną interpretację. Niektóre np. mówią nam coś o wyniku ostatniej instrukcji. Jest np. carry flag – wynik wyszedł poza liczbę, nastąpiło przeniesienie, parity flag – w wyniku jest parzysta liczba jedynek.

Instrukcja `jXX` oznacza skok warunkowy, gdzie warunek zależy od któregoś ze znaczników. Podczas instrukcji `sub A B` ustawiane są znaczniki, z których można np. wyciągnąć, która liczba jest większa (ważne jest to, że ustawią się różnie zależnie od tego, czy liczby mają znaki).

2. ABI Linuxa

2025-03-12

Podczas uruchamiania procesu najpierw sprawdzamy uprawnienia, potem alokujemy pamięć dla procesu (tworzymy przestrzeń adresową), kopiujemy sekcje inicjalizowane (np. kod do wykonania), ładujemy biblioteki linkowane dynamicznie, inicjalizujemy stos i rejestry, skaczemy na początek kodu (symbol `_start`). W C `_start` jest w bibliotece standardowej (tam jest inicjalizacja), która potem woła `main`.

Dane w przestrzeni adresowej są umieszczone w takiej kolejności (pierwsze punkty są na mniejszych adresach): kod i dane statyczne, sarta, biblioteki ładowane dynamicznie, stos.

Stos rośnie od góry do dołu a sarta od dołu do góry. Dlatego taka kolejność. O umieszczeniu bibliotek decyduje linker, przy starcie procesu daje się na sam dół, jak ładujemy coś w trakcie trwania procesu to idzie między stertę i stos.

Sterta istnieje dlatego, że stos jest zależny od wywołań funkcji. Pamięć, która ma przetrwać wywołania funkcji nie może na nim być.

W `/proc/<pid>/maps` mamy przestrzeń adresową (jej opis, w kolejności jak wyżej). Jeśli `<pid> -> self`, to widzimy swoją przestrzeń (procesu czytającego). Widzimy, że nad stosem jest coś jeszcze – to są jakieś informacje systemowe, o które proces może prosić. Są tam, żeby nie musiał robić wywołania systemowego.

Inicjalizacja stosu: na samej górze (czyli na najniższym adresie) `argc`, potem kolejne wskaźniki `argv[i]`, które wskazują na stos – dane są wpisane gdzieś wyżej. Potem jest 0 – widzimy, kiedy kończą się argumenty. Następnie wskaźnik na zmienne środowiskowe (napis z nimi). Potem 0, auxiliary entries, znowu 0 i na koniec rzeczy, na które wskaźniki były wcześniej.

Auxiliary entry to struktura, najpierw liczba oznaczająca rodzaj struktury, potem wartość (rozmiar stron, id użytkownika, jego grupa lub inne tego typu wartości).

Początkowy stan rejestrów:

- `rsp` – szczyt stosu.
- `rdx` – wskaźnik na funkcję do wywołania przed zakończeniem (raczej się nie używa).
- pozostałe – dowolnie.

Wywołania systemowe: `exit`, `open`, `read`, `write`, `brk`, `mmap`, `fork`, `exec`.

Robi się je za pomocą instrukcji `syscall` – nie ma argumentów, numer syscalla umieszczamy w `rax` (znajdujemy go w `/usr/include/asm/unistd_64.h`), kolejne argumenty w `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` (konwencja wywoływania funkcji jest lekko inna – `rcx` zamiast `r10`). Nadpisywane są argumenty oraz `rcx` i `r11`, wynik w `rax` (od `-4095` do `0`, ujemny kod błędu to `errno`). W rejestrze można umieścić wskaźnik do pamięci (w naszej przestrzeni adresowej), w ten sposób przekazujemy systemowi większe informacje. Jeśli system ma dać nam więcej danych, to przekazujemy mu wskaźnik do miejsca, gdzie ma pisać. Przekazywanie tych danych odbywa się za pomocą struktur (pisane kolejno po sobie dane o odpowiednich rozmiarach).

Możemy w Assemblerze zapisać dane na koniec różnych sekcji (statyczna alokacja):

```
1  ; segment kodu
2  SECTION .text
3  ; d - data, b - byte, dane bajt po bajcie
4  db "halo"
5
6  ; dane inicjalizowane
7  SECTION .data
8  ; word -- każda wartość to 2 bajty
9  dw 42, 54
10
11 ; dane nieinicjalizowane (to samo co wyżej, tylko bez zawartości
12   początkowej)
13 SECTION .bss
14 ; nie można db, bo ma nie być zawartości, możemy zarejestrować daną ilość
15   bajtów (lub innych jednostek, tutaj - 8 bajtów)
16 resq 4096
```

Dynamicznie możemy alokować pamięć:

- na stosie – przesuwamy wskaźnik na stos. Jeśli odwołamy się poniżej zaalokowanej pamięci stosu, to system nam go rozszerzy. Jest na to jakiś limit – nie możemy w ten sposób dostać bardzo dużo pamięci. Ta pamięć jest lokalna dla wywołania funkcji. Za to działanie ze stosem jest szybkie.
- syscall `brk` – działanie ze stertą, przesuwamy do góry adres końcowy sterty. Ten obszar jest zawsze spójny – jeśli zaalokujemy dwa bloki danych, to nie możemy zdealokować tego pierwszego bez zdealokowania drugiego. Dlatego proces musi sam zarządzać pamięcią (np. `malloc`).
- syscall `mmap` – mapuje plik do przestrzeni adresowej (w dowolne miejsce, jeśli nie podamy konkretnego adresu). Ten obszar może być współdzielony. Można też mapować strony pamięci bez

wskazywania pliku (dostajemy pamięć). Możemy te strony niezależnie zwalniać (lepiej niż na ster-
cie). Minus jest taki, że syscalls są wolne i zawsze dostajemy pełne strony (kwant po 4 KiB).

3. ABI C

2025-03-12

Pojawia się pojęcie wyrównywania (data alignment) – dokładnie się sztuczne dane, aby adresy były „okrągłe”. Pojedyncze zmienne nie muszą być wyrównane, ale warto to robić, bo jest to szybkie (procesor ma dostęp do pamięci za pomocą krótkich segmentów, jak nie ma wyrównania, to jedna zmienna może być pomiędzy segmentami). Wyjątek stanowią zmienne typu `packed` dla operacji SIMD (muszą być wyrównane zgodnie ze swoim rozmiarem).

W strukturach pola są wyrównane zgodnie ze swoim rozmiarem (`long double` do 16B), jeśli przed aktualnym polem występuje krótsze pole, to dodajemy wyrównanie do długości odpowiadającej aktualnemu polu. Do tego cała struktura jest wyrównana do tyłu bajtów ile wynosi największe wyrównanie jej składowej.

Podczas wywoływania funkcji niektóre rejestry muszą być zachowane:

- `rbp`, `rbx`, `r12-r15` są callee-saved, funkcja ma obowiązek je przywrócić.
- pozostałe są caller-saved, funkcja może je zmienić.
- flaga `DF` (direction flag) powinna być wyzerowana na wyjściu i wejściu do funkcji, pozostałe można zmieniać.
- bity kontrolne rejestru `mxcsr` i `x87 control word` muszą zostać zachowane (są używane przy operacjach zmiennoprzecinkowych).

Argumenty funkcji są przekazywane w rejestrach:

- ogólnego przeznaczenia (GPR), czyli w `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`.
- SSE, czyli 128-bitowe rejestry `xmm0` – `xmm7`.
- `al` – jest tam liczba argumentów przekazanych przez SSE.
- pozostałe są na stosie, wpisane od prawego do lewego (pierwszy na stos trafia najbardziej prawy argument).

Występują klasy argumentów, które mają przypisane to, w jakich rejestrach się znajdują:

- `INTEGER` – w rejestrach GPR.
- `SSE` – dolne 64 bity rejestru SSE.
- `SSEUP` – górne 64 bity rejestru SSE.
- `X87`, `X87UP` – przekazywane na stosie.
- `MEMORY` – również na stosie.

Liczby całkowite, `bool` i `void*` dostają `INTEGER`. Liczby zmiennoprzecinkowe i `__m64` dostają `SSE` (poza `long double`, to jest `X87`, `X87UP`). `__m128` i `__float128` dostają `SSE` i `SSEUP` (dzielą się na części), a `__m256` dostaje `SSE`, `SSEUP`, `SSEUP`, `SSEUP` (podział na 4 części).

Jeśli struktura ma ponad 16B lub jest wewnętrznie niewyrównana, to dostaje `MEMORY`. Inaczej elementy są klasyfikowane oddzielnie i łączone w grupy po 8B, dla których rozważamy przypadki: jeśli istnieje element `MEMORY`, całość dostaje `MEMORY`. Potem to samo, jeśli istnieje element `INTEGER`. Jeśli istnieje element `X87` lub `X87UP` całość dostaje `MEMORY`. W przeciwnym wypadku całość dostaje `SSE`. Jeśli któraś z grup jest `MEMORY`, to całość struktury dostaje `MEMORY`. Do tego w C++ klasy z nietrywialnym konstruktorem kopiującym lub destruktorom są przekazywane przez referencję jako `INTEGER`. Główny wniosek jest taki, że małe struktury mogą być przekazywane przez jeden rejestr – cała struktura może być `INTEGER` i zostać przekazana na raz.

Jeśli wynik funkcji jest klasy `MEMORY`, to musi ona otrzymać jako pierwszy argument adres miejsca na wynik, zwraca też ten adres w `rax`. `INTEGER` jest zwracany przez `rax`, a `SSE`, `SSEUP` przez `xmm0`,

X87, X87UP przez `st0`. Struktura, która może być zwracana przez rejestry trafia do `rax`, `rdx`, `xmm0`, `xmm1`.

Przed wywołaniem funkcji stos powinien zostać wyrównany do 16B. Na stos wrzucany jest adres powrotu, potem argumenty.

Po wartości `rsp` jest „red zone” – tam są wartości, które być może są na stosie, ale `rsp` nie zostało zmienione, aby to odzwierciedlać (procesor wsadził wartości, ale nie przesunął jeszcze `rsp`, żeby potem zrobić duże przesunięcie na raz). Debuggery wiedzą, żeby nie zmieniać rzeczy w tym miejscu, mimo że teoretycznie nie ma tam stosu.

Statyczne łączenie modułów jest łatwe, bo linker dostaje te moduły i wsadza je w odpowiednie miejsca podczas linkowania. W Assemblerze nazwy funkcji są takie same jak w C (w C++ trzeba dodać do deklaracji `extern "C"`).

```
1 ; import funkcji
2 EXTERN funkcja
3 call funkcja
4
5 ; oraz zmiennej
6 EXTERN zmienna
7 mov rax, [zmienna]
8
9 ; wprowadzenie symboli, aby były widoczne dla linkera
10 GLOBAL funkcja
11 GLOBAL zmienna
```

Biblioteki mogą być dołączane dynamicznie. Tak działają np. biblioteki współdzielone, bo nie chcemy, żeby każda binarka miała jej kopię w sobie. Wtedy segment kodu biblioteki może być współdzielony przez procesy (jest tylko jeden w RAM'ie), ale procesy widzą je pod różnymi adresami wirtualnymi. Do tego sekcje danych są oddzielne dla procesów korzystających z biblioteki. Zatem biblioteka powinna działać niezależnie od swojej pozycji (Position Independent Code), w szczególności nie mieć żadnego zhardcodowanego adresu w swoim kodzie.

Position Independent Code w trybie 64-bitowym wykorzystuje adresowanie względne. Wszystkie sekcje biblioteki są ładowane w spójnym bloku pamięci, odwoływanie się do danych wykorzystuje adresowanie względem RIP (`mov rax, [rel zmienna]`)

```
1 ; przy dynamicznym eksportowaniu modułów podajemy typ -- czy jest to zmienna,
   ; czy funkcja
2 GLOBAL symbol:function
3
4 ; przy eksportowaniu danych możemy dodatkowo wskazać ich długość
5 GLOBAL tablica:data tablica.end-tablica
6 tablica: resd 128
7 .end:
```

Biblioteka współdzielona zostanie wsadzona w pewne nieznane miejsce przestrzeni adresowej. Dlatego w pewnym ustalonym miejscu istnieje Procedure Linkage Table (PLT) – zawiera wskaźniki do funkcji importowanych dynamicznie. Aby wywołać taką funkcję robimy `call printf wrt ..plt` – mówimy, że symbol jest w sekcji `..plt`. Pod takim symbolem znajduje się kod dynamic-linkera, który w razie potrzeby ładuje bibliotekę i uzupełnia referencje, a potem skacze do odpowiedniego kodu. Podobnie działają eksportowane dynamicznie dane. Mamy Global Offset Table (GOT), w której są adresy danych zewnętrznych.

```
1 EXTERN variable
2 mov rsi, _GLOBAL_OFFSET_TABLE_
3 mov rsi, [rsi + variable wrt ..got] ; znajdujemy adres i pod niego patrzymy
```

4. ABI C++

2025-03-26

W C nie może być dwóch funkcji o tej samej nazwie i różnych typach. W C++ mogą być. W Assembly muszą dostać różne nazwy. Dlatego występuje name mangling – dodajemy typ do nazwy funkcji. Jeśli chcemy tego uniknąć możemy użyć ABI języka C dodając `extern "C"` do deklaracji funkcji, ale wtedy nie możemy przeładowywać jej nazwy.

Aby zobaczyć symbole w pliku robimy `objdump -t file.o`. Tłumaczenie zmanglowanych nazw na coś czytelnego za pomocą `c++filt` – bierze na wejście jakiś tekst i demangluje wszystko, co wygląda na symbol.

Referencje to w kodzie maszynowym normalne wskaźniki. Metody obiektów to zwykłe funkcje, które jako pierwszy argument dostają wskaźnik na `this`. Enkapsulacja istnieje tylko na poziomie C++.

Dane w strukturach są układane w taki sposób, że najpierw pomijamy wszystkie funkcje i umieszczamy zmienne jak w C. Metody niewirtualne to funkcje niezależne od struktur, leżą osobno, ze strukturą łączy je tylko ich nazwa.

Przy metodach wirtualnych (mogących się zmieniać zależnie od typu dynamicznego obiektu) trzeba się bardziej wysilić. W językach dynamicznych typu Python metody dynamiczne są po prostu wskaźnikami w obiekcie. Wtedy jednak każda instancja musi mieć swój osobny wskaźnik do każdej metody – duże zużycie pamięci, a na pewno można lepiej, bo w C++ nie określamy tej metody na poziomie instancji, tylko klasy – znając dokładny typ obiektu znamy implementację wszystkich metod wirtualnych. Dlatego w instancji jest wskaźnik do tabeli metod wirtualnych (vtable), w niej są wskaźniki do implementacji metod. Taka tabela jest jedna na całą klasę.

Pierwszym wpisem w strukturze jest wskaźnik na vtable. To jest wskaźnik na opis pierwszej metody wirtualnej, czyli zazwyczaj `vtable+16` – pod `vtable` znajduje się `top_offset` (informacja o tym, jak daleko wskaźnik w strukturze wskazujący na vtable znajduje się od początku struktury), a pod `vtable+8` jest `typeinfo` – identyfikuje dokładny typ obiektu.

Jednokrotne dziedziczenie działa prosto – do struktury i vtable typu rodzica dokładamy na koniec trochę nowych rzeczy. Ciężiej jest przy wielokrotnym dziedziczeniu. Mając `C : A, B` musimy tak ułożyć dane, aby dało się patrzeć na obiekt typu C jak na obiekt typu B. W tym celu w strukturze i vtable najpierw umieszczamy rzeczy pochodzące z A, potem w strukturze umieszczany jest drugi wskaźnik na vtable – tym razem na jej późniejszy fragment, w którym zaczynają się metody B (a nad nimi jest `top_offset` i `typeinfo`). Po nim następują dane obiektu B, a na końcu są dane obiektu C (tak jak przy jednokrotnym dziedziczeniu).

Przy przesłanianiu funkcji wirtualnych jest problem – jeśli nadpiszemy implementację w klasie-dziecku, to jako swój pierwszy argument będzie ona oczekiwać wskaźnika na obiekt tego typu, a nie typu rodzica, co jest problematyczne, gdy wykonujemy ją przez obiekt zcastowany na typ rodzica. Aby sobie z tym poradzić zastępujemy wskaźnik na taką funkcję w vtable przez tak zwany thunk – fragment kodu, który przesunie wskaźnik na obiekt do tego, którego oczekuje implementacja wywoływanej metody, a dopiero potem zawoła metodę. Thunki muszą być różne dla różnych funkcji, bo muszą mieć w sobie zhardcodowane wartości (a nie po prostu korzystać z `top_offset`) – metoda może być przesłonięta na innym poziomie niż najwyższy.

W przypadku wielokrotnego dziedziczenia po tym samym typie trzymamy kilka kopii podobiektów tego typu. Inaczej jest przy dziedziczeniu wirtualnym – wszystkie wirtualne dziedziczenia po danym typie są zaspokajane przez wsadzenie struktury tego typu na koniec obiektu. W szczególności można dziedziczyć po jakimś typie wirtualnie i niewirtualnie, wtedy niewirtualne dziedziczenie dostanie swój osobny obiekt tak jak zazwyczaj.

To, gdzie są podobiekty wirtualne zależy od struktury całej klasy (bo są wsadzane na koniec). Zatem informacja o tym musi być zawarta w vtable. Nad każdym `top_offset` umieszczamy dodatkowo `vbase_offset` dla każdej klasy wirtualnej (w kolejności preorder na listach dziedziczenia) – informacja o tym, o ile trzeba przesunąć wskaźnik, aby dostać obiekt danego typu.

Podobnie thunki nie mogą mieć w sobie stałej jak przedtem, więc do vtable dodajemy `vcall_offset` (o ile trzeba przesunąć wskaźnik na obiekt klasy wirtualnej, aby otrzymać wskaźnik na obiekt typu, który nadpisuje metodę wirtualną). Umieszczamy go nad `vbase_offset` klasy wirtualnej. Każda metoda wirtualna dostaje swój.